

SMART NOTEBOOK

Phase 1 Blueprint

Postgres-as-Queue + Python CLI Worker Architecture

Version 1.0 | February 2026

Target: Staff/Principal Engineer Interview Portfolio

Document Purpose

This blueprint defines the Phase 1 implementation of Smart Notebook — a personal knowledge base that demonstrates distributed systems patterns suitable for L6/L7 interviews. Phase 1 focuses on a working end-to-end vertical slice using a Postgres-as-Queue pattern with a Python CLI worker, optimized for an 8GB M2 Mac.

1. Executive Summary

Smart Notebook is a personal knowledge base where you upload documents (PDFs, markdown, text), the system ingests, chunks, and indexes them using AI, and you can ask natural-language questions with cited, grounded answers.

Phase 1 delivers a working end-to-end vertical slice: upload a document, have it ingested asynchronously by a Python worker, stored as vector embeddings, and queried with RAG-powered answers. The architecture is deliberately simple but production-patterned.

Phase 1 Scope

Spring Boot API → Postgres task queue → Python CLI worker → Ollama embeddings → pgvector search → RAG answers

Everything runs locally on an 8GB M2 MacBook. No cloud dependencies. No message broker overhead.

Interview Positioning

Phase 1 alone covers 4+ Staff-level talking points: polyglot microservice boundary, async task processing with exactly-once semantics (SELECT FOR UPDATE SKIP LOCKED), vendor-agnostic model abstraction, and vector search with RAG. Each decision maps to a specific architectural trade-off discussion.

2. Architecture Decision: Why Option 1

Three approaches were evaluated for the Python ingestion worker. Option 1 (Postgres-as-Queue + CLI Worker) was selected for Phase 1. Here is the reasoning:

2.1 Options Evaluated

Option	Description	Verdict
Option 1: Postgres-as-Queue + CLI Worker	Spring Boot writes tasks to a Postgres table. Python script polls using SELECT FOR UPDATE SKIP LOCKED. No additional infrastructure.	SELECTED for Phase 1
Option 2: Flask/FastAPI Sidecar	Always-on Python HTTP service alongside Spring Boot. Communicates via REST or gRPC.	Deferred to Phase 2+
Option 3: Java-only (Spring AI / LangChain4j)	Eliminate Python entirely. Use Java-native AI libraries for ingestion, embedding, and RAG.	Considered for simplification

2.2 Why NOT Option 2 (Flask/FastAPI Sidecar)

Rejected for Phase 1

An always-on Python service adds unnecessary resource pressure on an 8GB M2 MacBook.

- **RAM pressure:** Running Spring Boot (~300-500MB) + FastAPI/Gunicorn (~150-200MB) + Ollama (~2-4GB) + Postgres (~100MB) simultaneously on 8GB RAM leaves almost no headroom. Swap thrashing kills developer velocity.
- **Idle resource waste:** The Python service sits idle 95% of the time waiting for ingestion tasks. A CLI worker that only runs when there is work is far more efficient for local development.
- **Operational overhead:** Two long-running services means two processes to manage, two sets of logs, two health checks. For a solo developer on a local machine, this is unnecessary friction.
- **Premature coupling:** An HTTP sidecar implies a synchronous contract between services. The Postgres queue naturally gives you async decoupling, which is the production pattern you actually want.
- **Phase 2 upgrade path:** When ready for MCP server integration or real-time processing, upgrading the Python worker to a FastAPI service is a clean, incremental change — the task queue contract stays the same.

2.3 Why NOT Option 3 (Java-Only)

Rejected for Phase 1

Eliminating Python removes a key architectural talking point and limits AI ecosystem access.

- **Interview signal loss:** The polyglot microservice boundary (Java API + Python worker) is a direct talking point for Staff/Principal interviews. It demonstrates that you can make deliberate language choices based on ecosystem strengths — Java for the API/orchestration layer, Python for the AI/ML pipeline.
- **Ecosystem maturity:** Python's AI ecosystem (LangChain, sentence-transformers, PyPDF, tiktoken, unstructured) is 2-3 years ahead of Java equivalents. Spring AI and LangChain4j are maturing fast but still lack parity for chunking strategies, PDF extraction quality, and model provider coverage.
- **Ollama integration:** While both Java and Python can call Ollama's HTTP API, Python libraries like langchain-ollama provide higher-level abstractions for embedding batching, retry logic, and model switching that would need to be hand-built in Java.
- **Future flexibility:** Phase 3+ features (council-of-models, recursive learning, A2A agents) lean heavily on Python ML tooling. Starting with Python in Phase 1 means the ingestion pipeline is already in the right language for future phases.

- **Design doc value:** You can still document the Java-only option as an alternative in your design doc and explain the trade-off. This shows architectural judgment — knowing when to go polyglot vs. monoglot is exactly the kind of decision L6/L7 interviewers probe.

2.4 Why Option 1 WINS for Phase 1

Key Advantages

Zero additional infrastructure — Postgres you already have serves as the task queue.

Worker only consumes resources when processing — critical for 8GB RAM constraint.

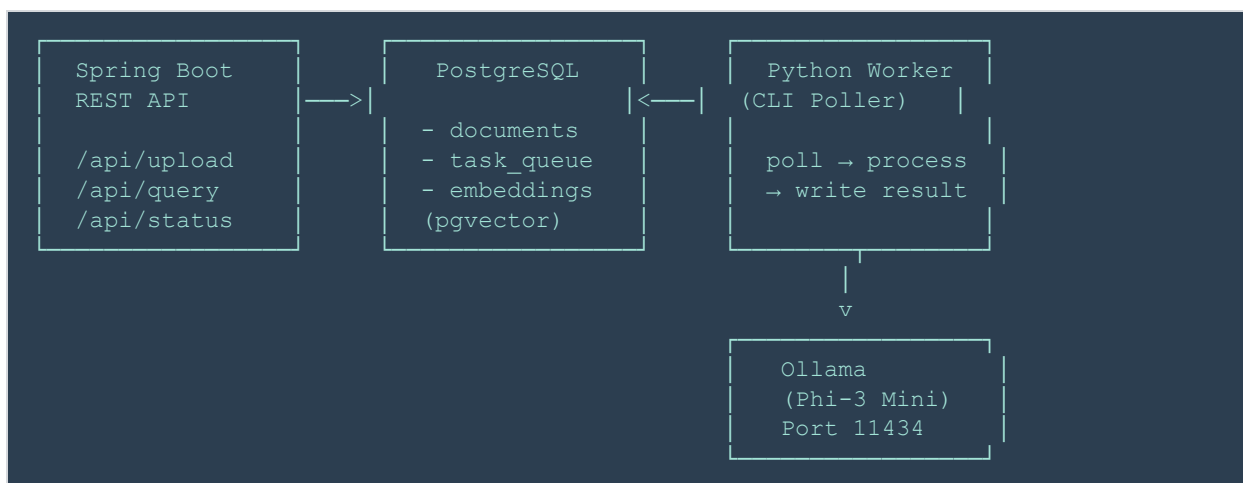
SELECT FOR UPDATE SKIP LOCKED is the production pattern used by systems like Que, PGMQ, and Temporal.

Clean upgrade path: swap Postgres polling for SQS/Kafka later without changing the worker contract.

Demonstrates exactly-once processing semantics — a common Staff interview question.

3. Phase 1 System Architecture

The system has four components, all running locally:



3.1 Data Flow

Step	Action	Component
1	User uploads a document via REST API	Spring Boot

2	API stores raw document + creates PENDING task in ingestion_tasks table	Spring Boot → Postgres
3	Python worker polls for PENDING tasks using SELECT FOR UPDATE SKIP LOCKED	Python Worker → Postgres
4	Worker claims task (atomically sets status = PROCESSING)	Python Worker
5	Worker extracts text, chunks it, generates embeddings via Ollama	Python Worker → Ollama
6	Worker stores embeddings in pgvector + updates task status = COMPLETED	Python Worker → Postgres
7	User queries via /api/query. Spring Boot does vector similarity search + RAG	Spring Boot → Postgres → Ollama
8	RAG-powered answer with citations returned to user	Spring Boot

4. Database Schema

Three core tables power Phase 1. All live in a single Postgres instance with the pgvector extension.

4.1 documents

Stores the raw uploaded documents and metadata.

```
CREATE TABLE documents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       VARCHAR(500) NOT NULL,
  content_type VARCHAR(50) NOT NULL,          -- pdf, markdown, text
  raw_content TEXT,                          -- extracted text
  file_path   VARCHAR(1000),                  -- original file location
  status      VARCHAR(20) DEFAULT 'UPLOADED',
  metadata    JSONB DEFAULT '{}',
  created_at  TIMESTAMP DEFAULT NOW(),
  updated_at  TIMESTAMP DEFAULT NOW()
);
```

4.2 ingestion_tasks (Task Queue)

The heart of the async processing pattern. This table acts as a durable, transactional task queue.

```
CREATE TABLE ingestion_tasks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

document_id  UUID NOT NULL REFERENCES documents(id),
status       VARCHAR(20) DEFAULT 'PENDING',
             -- PENDING | PROCESSING | COMPLETED | FAILED
payload      JSONB NOT NULL,
result       JSONB,
error_message TEXT,
retry_count  INT DEFAULT 0,
max_retries  INT DEFAULT 3,
created_at   TIMESTAMP DEFAULT NOW(),
updated_at   TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_tasks_status ON ingestion_tasks(status);
CREATE INDEX idx_tasks_document ON ingestion_tasks(document_id);

```

Interview Talking Point — Postgres as Queue

"We use SELECT FOR UPDATE SKIP LOCKED for exactly-once task claiming. This is the same pattern used by Que (Ruby), PGMQ (Tembo), and even parts of Temporal. It gives us transactional guarantees without a separate broker, which is critical for a system where ingestion failures must not lose user data. The trade-off vs. SQS/Kafka is polling overhead and horizontal scale ceiling — acceptable for our workload."

4.3 document_chunks (Vector Store)

Stores chunked text with vector embeddings for similarity search.

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES documents(id),
  chunk_index INT NOT NULL,
  content     TEXT NOT NULL,
  embedding   vector(384),      -- Phi-3 Mini embedding dimension
  metadata    JSONB DEFAULT '{}',
  created_at  TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_chunks_document ON document_chunks(document_id);
CREATE INDEX idx_chunks_embedding ON document_chunks
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

5. Spring Boot API Layer

The API layer is intentionally thin in Phase 1. Three endpoints cover the complete workflow.

5.1 REST Endpoints

Method	Path	Description	Returns
POST	/api/documents/upload	Upload a document (multipart file)	document_id + task_id
GET	/api/tasks/{taskId}/status	Poll ingestion task status	status, progress, errors
POST	/api/query	Ask a question against ingested docs	answer + citations + confidence

5.2 Key Java Interfaces

Model Gateway (Vendor Abstraction)

Even in Phase 1, the model gateway abstracts the LLM provider. This allows swapping Ollama for Anthropic/OpenAI without changing business logic.

```
public interface ModelGateway {
    CompletionResponse complete(CompletionRequest request);
    EmbeddingResponse embed(EmbeddingRequest request);
    ModelHealth health();
}

@Component
@Profile("local")
public class OllamaModelGateway implements ModelGateway {
    // Phase 1: talks to localhost:11434
    // Phase 2: add AnthropicModelGateway, OpenAIModelGateway
}
```

Query Service (RAG Pipeline)

```
@Service
public class QueryService {

    private final ModelGateway modelGateway;
    private final VectorSearchRepository vectorSearch;

    public QueryResponse query(String question) {
        // 1. Embed the question
        var qEmbedding = modelGateway.embed(
            new EmbeddingRequest(question));

        // 2. Vector similarity search (top-k chunks)
        var relevantChunks = vectorSearch
```

```

        .findSimilar(qEmbedding.vector(), 5);

// 3. Build RAG prompt with retrieved context
var prompt = buildRagPrompt(question, relevantChunks);

// 4. Get completion with citations
var answer = modelGateway.complete(
    new CompletionRequest(prompt));

return new QueryResponse(
    answer.text(),
    extractCitations(answer, relevantChunks),
    answer.confidence()
);
}
}

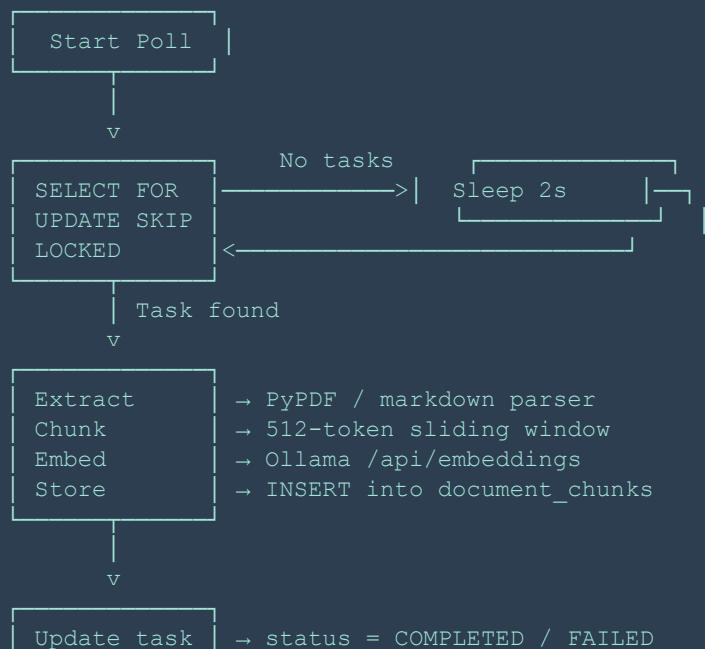
```

6. Python CLI Worker

The worker is a single-file Python script that polls Postgres for pending tasks, processes them, and writes results back. It runs only when invoked from the terminal.

6.1 Worker Lifecycle

Terminal 2: \$ python worker.py



6.2 Exactly-Once Claim Pattern

The critical SQL pattern that ensures no two workers ever process the same task:

```
UPDATE ingestion_tasks
SET status = 'PROCESSING', updated_at = NOW()
WHERE id = (
  SELECT id FROM ingestion_tasks
  WHERE status = 'PENDING'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING *;
```

Why FOR UPDATE SKIP LOCKED?

FOR UPDATE — locks the row so no other transaction can claim it.

SKIP LOCKED — if a row is already locked, skip it instead of blocking.

Combined, this gives exactly-once semantics without distributed locks or brokers.

The trade-off: polling interval (2s) adds latency vs. push-based queues.

At our scale (< 100 docs/day), this is a non-issue.

6.3 Retry & Error Handling

Scenario	Behavior	Interview Angle
Worker crashes mid-task	Task stays PROCESSING. Stale task reaper (cron) resets to PENDING after 5min timeout.	Failure recovery without distributed coordination
Ollama unavailable	Catch exception, increment retry_count, set FAILED if retries exhausted	Graceful degradation, circuit breaker pattern
Malformed PDF	Log error with document_id, set FAILED with error_message	Observability, structured error logging
Worker restart	Picks up next PENDING task. No state lost.	Stateless worker design

6.4 Stale Task Reaper

A background thread or cron job that reclaims stuck tasks:

```

-- Reset tasks stuck in PROCESSING for > 5 minutes
UPDATE ingestion_tasks
SET status = 'PENDING',
    retry_count = retry_count + 1,
    updated_at = NOW()
WHERE status = 'PROCESSING'
    AND updated_at < NOW() - INTERVAL '5 minutes'
    AND retry_count < max_retries;

-- Mark exhausted retries as permanently FAILED
UPDATE ingestion_tasks
SET status = 'FAILED',
    error_message = 'Max retries exceeded',
    updated_at = NOW()
WHERE status = 'PROCESSING'
    AND updated_at < NOW() - INTERVAL '5 minutes'
    AND retry_count >= max_retries;

```

7. Vector Search & RAG

7.1 Embedding Pipeline

Stage	Implementation	Config
Text Extraction	PyPDF2 for PDFs, raw read for .md/.txt	UTF-8 normalized
Chunking	Sliding window, 512 tokens, 50-token overlap	Configurable via env vars
Embedding	Ollama /api/embeddings with Phi-3 Mini	384-dim vectors
Storage	INSERT into document_chunks with pgvector	IVFFlat index, 100 lists

7.2 Query-Time RAG

When a user asks a question, the query service follows a standard RAG pattern:

```

-- pgvector similarity search (cosine distance)
SELECT dc.content, dc.metadata,
       d.title,
       1 - (dc.embedding <=> $1::vector) AS similarity
FROM document_chunks dc
JOIN documents d ON dc.document_id = d.id
ORDER BY dc.embedding <=> $1::vector
LIMIT 5;

```

RAG Prompt Template

You are a knowledge assistant. Answer based ONLY on the provided context.
 If the context doesn't contain the answer, say "I don't have enough information."
 Always cite which document and section your answer comes from.

Context: {retrieved_chunks}

Question: {user_question}

Answer with citations:

8. Project Structure

```

smart-notebook/
├── api/                                # Spring Boot application
│   ├── src/main/java/
│   │   └── com/smartnotebook/
│   │       ├── controller/            # REST endpoints
│   │       ├── service/               # Business logic
│   │       │   ├── IngestionService.java
│   │       │   └── QueryService.java
│   │       ├── model/                 # Domain objects
│   │       ├── gateway/               # Model abstraction
│   │       │   ├── ModelGateway.java
│   │       │   └── OllamaModelGateway.java
│   │       └── repository/            # Data access
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── db/migration/              # Flyway migrations
│   └── pom.xml
├── worker/                             # Python ingestion worker
│   ├── worker.py                     # Main poll loop
│   ├── processor.py                  # Extract / chunk / embed
│   ├── requirements.txt
│   └── .env                           # DB connection, Ollama URL
├── db/                                 # Database scripts
│   ├── init.sql                      # Schema + pgvector extension
│   └── seed.sql                      # Test data (optional)
├── docker-compose.yml                 # Postgres + pgvector only
├── Makefile                           # Dev commands
└── README.md
  
```

9. Local Development Setup

Optimized for an 8GB M2 MacBook. Only Postgres runs in Docker; Spring Boot and the Python worker run natively.

9.1 docker-compose.yml

```
version: '3.8'
services:
  postgres:
    image: pgvector/pgvector:pg16
    ports:
      - '5432:5432'
    environment:
      POSTGRES_DB: smartnotebook
      POSTGRES_USER: notebook
      POSTGRES_PASSWORD: notebook_dev
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    deploy:
      resources:
        limits:
          memory: 256M

volumes:
  pgdata:
```

9.2 Makefile

```
# Start Postgres
db-up:
docker compose up -d postgres

# Start Spring Boot API
api:
cd api && ./mvnw spring-boot:run

# Start Python worker
worker:
cd worker && python worker.py

# Start Ollama (if not running)
ollama:
ollama serve &
ollama pull phi3:mini

# Full stack (run in separate terminals)
dev: db-up
@echo 'Run in separate terminals:'
@echo '  make api'
@echo '  make worker'
@echo '  make ollama'
```

9.3 Memory Budget (8GB M2)

Component	Estimated RAM	Notes
macOS + Desktop	~2.0 GB	Base OS overhead
Ollama (Phi-3 Mini)	~2.5 GB	Quantized model, largest consumer
Postgres (Docker)	~256 MB	Capped via Docker memory limit
Spring Boot	~400 MB	JVM with -Xmx384m
Python Worker	~100 MB	Only when actively processing
IDE / Browser	~1.5 GB	IntelliJ or VS Code
Headroom	~1.2 GB	Swap buffer

RAM Optimization Tips

Set JVM max heap: `JAVA_OPTS="-Xmx384m"` in `application.yml`
 Use Ollama quantized models (Q4_K_M) to reduce model memory footprint
 Python worker is ephemeral — only consumes RAM during active processing
 Close IntelliJ when running full stack; use VS Code for lighter footprint

10. Implementation Timeline

Phase 1 targets 2 weeks of focused evenings/weekends. Each milestone produces a testable increment.

Day	Milestone	Deliverable
Day 1	Project scaffolding	Monorepo structure, docker-compose, Flyway migrations for 3 tables
Day 2	Model Gateway	ModelGateway interface + OllamaModelGateway. Verify embedding call works.
Day 3	Upload API + Task Queue	POST /api/documents/upload endpoint. Document saved, task enqueued.
Day 4	Python Worker v1	worker.py with poll loop, task claiming, basic text extraction + chunking
Day 5	Embedding Integration	Worker calls Ollama, stores vectors in document_chunks via pgvector
Day 6	Query API + RAG	POST /api/query with vector search + RAG prompt. First end-to-end answer.
Day 7	Error Handling	Retry logic, stale task reaper, structured error logging

Day 8-9	Polish + Testing	Integration tests, edge cases (large PDF, empty file, Ollama down)
Day 10	Demo Recording	Screen recording of full workflow: upload → ingest → query → cited answer

11. Evolution Path: Phase 1 → Phase 4

Phase 1 is designed to evolve cleanly. Here is how each future phase builds on Phase 1 without rewriting it:

Phase	Adds	What Changes in Phase 1
Phase 2: MCP + Multi-Model	MCP server interface, confidence-based model router, FastAPI sidecar upgrade	Python worker becomes a FastAPI service. ModelGateway gets Anthropic/OpenAI providers. Task queue optionally migrates to Redis/SQS.
Phase 3: Feedback Loop	Answer logging, groundedness scoring, patch plan generation	New tables (query_logs, feedback_scores). QueryService adds logging. Cheap LLM grades answers.
Phase 4: Council + A2A	Council-of-models for ingestion, Agent Cards, A2A interfaces	Ingestion pipeline splits into multiple agents (extractor, tagger, critic). Each gets an Agent Card for discovery.

Key Principle

Each phase produces a working, demo-able system. Don't move to Phase 2 until Phase 1 works end-to-end. The Postgres-as-queue + ModelGateway + pgvector + async ingestion from Phase 1 alone is already a strong interview talking point.

12. Interview Discussion Map

How to use Phase 1 in Staff/Principal interviews:

Interview Question	Phase 1 Answer	Depth Signal
"How do you handle async processing?"	Postgres-as-queue with FOR UPDATE SKIP LOCKED. Exactly-once semantics without a broker.	Compare with SQS/Kafka trade-offs. Discuss polling vs push, visibility timeout patterns.
"Why polyglot? Why not all Java?"	Java for API/orchestration. Python for AI/ML pipeline. Each language used for its ecosystem strength.	Discuss ecosystem maturity, hiring, operational overhead. Reference Spring AI as future option.

"How would you scale this?"	Workers are stateless — horizontal scale by adding instances. Queue pattern supports competing consumers.	Discuss when to migrate from Postgres queue to SQS/Kafka. Partitioning strategies. Back-pressure.
"What about failures?"	Retry with exponential backoff. Stale task reaper for crashed workers. Dead letter queue for poison messages.	Discuss idempotency, exactly-once vs at-least-once, compensating transactions.
"How do you abstract the LLM layer?"	ModelGateway interface with provider-specific implementations. Profile-based provider selection.	Discuss vendor lock-in, cost optimization, A/B testing models, graceful degradation.
"Tell me about your RAG implementation."	pgvector cosine similarity. Top-k retrieval. Prompt engineering for grounded, cited answers.	Discuss chunking strategies, embedding models, re-ranking, hybrid search (BM25 + vector).

13. Risks & Mitigations

Risk	Impact	Mitigation
Ollama OOM on 8GB Mac	High — blocks all AI features	Use quantized models (Q4_K_M). Monitor with Activity Monitor. Kill Ollama when not needed.
pgvector index too slow	Medium — slow queries	IVFFlat with 100 lists is fine for < 10K chunks. Upgrade to HNSW if needed.
PDF extraction quality	Medium — bad chunks = bad answers	Use PyPDF2 + fallback to pdfplumber. Log extraction quality metrics.
Scope creep into Phase 2	High — never ships Phase 1	Strict scope boundary. Phase 1 = upload + ingest + query. Nothing else.
Embedding dimension mismatch	High — silent failures	Validate embedding dimensions match pgvector column. Add assertion in worker.

Scope Lock

Phase 1 is DONE when: you can upload a PDF, it gets ingested asynchronously, and you can ask a question and get a cited answer. That is the complete definition of done. Everything else is Phase 2+.